

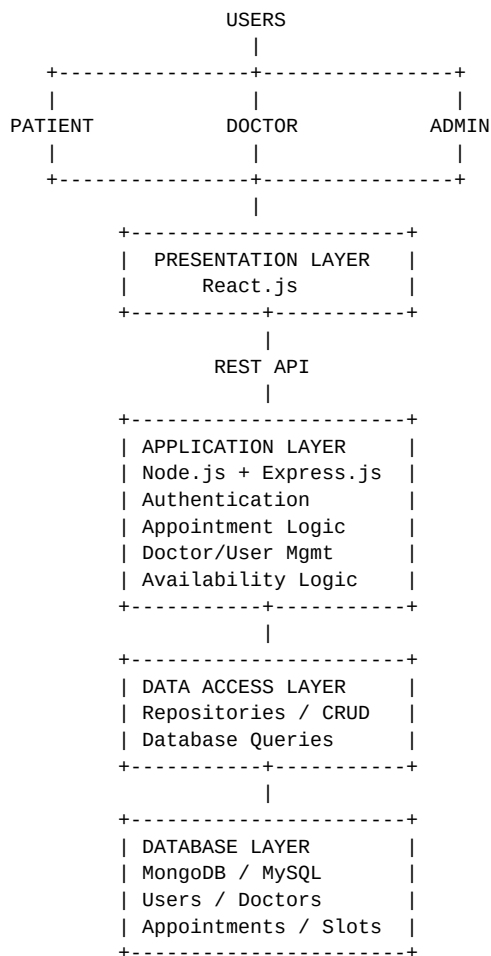
Architecture Design Report

Online Doctor Appointment System

1. Architecture Overview

The Online Doctor Appointment System follows a Layered Architecture in which the application is divided into logical layers. Each layer is responsible for a specific set of operations and communicates with the layer directly above or below it. The proposed system uses four layers: Presentation Layer, Application/Business Logic Layer, Data Access Layer, and Database Layer.

2. Layered Architecture



3. Presentation Layer

The Presentation Layer is the topmost layer and provides the interface through which patients, doctors, and administrators interact with the system.

Technologies: React.js, HTML, CSS, JavaScript, and optionally Tailwind CSS.

Responsibilities: registration and login, patient/doctor/admin dashboards, doctor search, doctor profiles, appointment booking, cancellation, appointment history, availability display, forms, and user feedback.

4. Application / Business Logic Layer

The Application Layer is the core of the system. It processes requests from the Presentation Layer and applies the business rules of the application.

Technologies: Node.js and Express.js.

Responsibilities: authentication, authorization, user management, doctor management, appointment management, availability management, validation, prevention of appointment conflicts, and role-based access control.

Appointment booking flow:

```
Patient requests appointment
|
Check authentication
|
Check doctor exists
|
Check selected time slot
|
Check slot availability/conflict
|
Create appointment
|
Return confirmation
```

5. Data Access Layer

The Data Access Layer acts as an intermediate layer between the Business Logic Layer and the database. It handles database communication and provides reusable methods for retrieving and modifying data.

Responsibilities: database queries, CRUD operations, user repository, doctor repository, appointment repository, availability queries, and data validation.

```
Business Layer
|
Appointment Repository
|
Database Query
|
Database
```

6. Database Layer

The Database Layer stores all persistent information required by the application. The project can use either MySQL or MongoDB.

Entity	Example Fields
Users	user_id, name, email, password, phone, role
Doctors	doctor_id, name, specialization, qualification, experience, location
Appointments	appointment_id, patient_id, doctor_id, date, time, status, reason
Availability	availability_id, doctor_id, date, start_time, end_time, status

7. Communication Between Layers

```
Presentation Layer
|
| HTTP / REST API
v
Application Layer
|
| Service / Function Calls
v
Data Access Layer
|
| Database Queries
```

8. Role-Based Architecture

Role	Main Functions
Patient	Register/login, search doctors, view profiles, check slots, book/cancel appointments, view history
Doctor	Login, manage profile, set availability, view appointments, manage schedule
Admin	Manage patients and doctors, monitor appointments, add/remove doctor information, manage system data

9. Example System Flow

When a patient wants to book an appointment with a cardiologist, the following flow takes place:

```
Patient
|
React Search Interface
|
GET /api/doctors?specialization=Cardiology
|
Doctor Controller
|
Doctor Service
|
Doctor Repository
|
Database
|
Available doctors returned to patient
```

Appointment Booking Flow

```
Patient selects slot
|
POST /api/appointments
|
Appointment Controller
|
Appointment Service
|
Check slot availability
|
Create appointment
|
Database
|
Confirmation returned to patient
```

10. Security Architecture

Because the application handles user and appointment information, security is an important part of the design.

- Authentication using email/password.
- Secure password hashing rather than storing plain-text passwords.
- JWT-based authentication for API access.
- Role-based authorization for Patient, Doctor, and Admin.
- Input validation and secure API access.
- HTTPS during deployment.
- Token/session expiration and protection against unauthorized access.

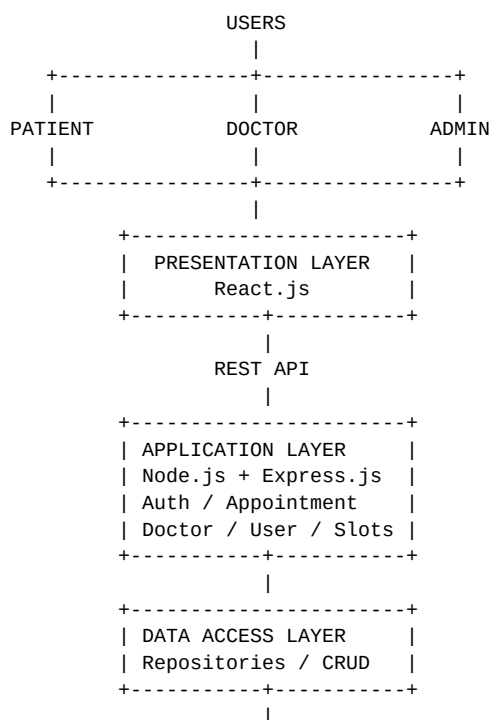
11. Advantages of Layered Architecture

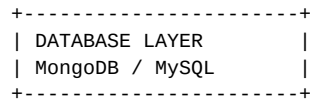
Advantage	Description
Separation of Concerns	Each layer has a specific responsibility.
Maintainability	Changes can be made within a layer with limited impact on other layers.
Scalability	Components can be improved or scaled as usage grows.
Reusability	Services and data-access components can be reused.
Testability	Individual layers can be tested independently.
Security	Authentication and authorization can be centralized in the backend.

12. Proposed Technology Stack

Layer/Component	Technology
Presentation	React.js, HTML, CSS, JavaScript
API/Backend	Node.js, Express.js
Business Logic	Node.js / Express.js Services
Data Access	Mongoose / MySQL Driver / ORM
Database	MongoDB or MySQL
Authentication	JWT + Password Hashing
Version Control	Git & GitHub
Development Environment	VS Code
Deployment	Web Server / Cloud Platform

13. Overall Architecture





14. Conclusion

The proposed Layered Architecture provides a structured design for the Online Doctor Appointment System. By separating the presentation, business logic, data access, and database layers, the system becomes easier to develop, test, maintain, and scale. The architecture supports patient registration, doctor search, availability management, appointment booking and cancellation, appointment history, doctor schedule management, and administrative functions. It can also be extended in the future to support online payments, video consultations, notifications, digital prescriptions, medical records, AI-based recommendations, and mobile applications.